

REGISTRO BACK END - SEMANA 08

Aula 01 - Código testado no Python:

Sistema Monolítico

```
class Ecommerce:
    def __init__(self):
        self.products = ["Laptop"]

    def order(self, product):
        if product in self.products:
            return "Pedido realizado!"
        return "Produto indisponível."
```

```
system = Ecommerce()
print(system.order("Laptop"))
```

Sistema com Microserviços

```
class ProductService:
    def __init__(self):
        self.products = ["Laptop"]

    def available(self, product):
        return product in self.products
```

```
class OrderService:
    def order(self, product, product_service):
        if product_service.available(product):
            return "Pedido realizado!"
        return "Produto indisponível."
```

```
product_service = ProductService()
order_service = OrderService()
```

```
print(order_service.order("Laptop", product_service))
```

Exemplo de uso

```
print("\n=== Sistema com Microserviços ===")
```

```

user_service = UserService()
product_service = ProductService()
order_service = OrderService()

user_service.add_user("Alice")
product_service.add_product("Laptop")

resultado = order_service.place_order(
    "Alice",
    "Laptop",
    product_service
)

print(resultado)

print("Usuários:", user_service.users)
print("Produtos:", product_service.products)
print("Pedidos:", order_service.orders)

```

Reflexão

A atividade mostrou que o sistema monolítico é mais simples, mas os microsserviços deixam o código mais organizado e fácil de expandir.

Perguntas

1. Como a implementação de microsserviços mudou a maneira como você pensou sobre a divisão do código?

Mostrou que dividir o sistema em partes menores deixa o código mais organizado.

2. Quais desafios surgiram ao tentar refatorar o sistema para microsserviços?

O principal desafio foi separar corretamente as funções de cada serviço.

3. Como os padrões de design ajudaram a estruturar melhor o código em ambas as arquiteturas?

Eles ajudaram a organizar melhor o código e facilitaram a manutenção do sistema.

Aula 02 - Código testado em Python:

Loja Monolítica

```

class Store:
    def __init__(self):
        self.products = ["Laptop"]

```

```
def order(self, product):
    if product in self.products:
        return "Pedido realizado!"
    return "Produto indisponível."
```

```
store = Store()
print(store.order("Laptop"))
```

Loja com Microserviços

```
class ProductService:
    def __init__(self):
        self.products = ["Laptop"]

    def available(self, product):
        return product in self.products
```

```
class OrderService:
    def order(self, product, product_service):
        if product_service.available(product):
            return "Pedido realizado!"
        return "Produto indisponível."
```

```
product_service = ProductService()
order_service = OrderService()

print(order_service.order("Laptop", product_service))
```

Reflexão

A atividade mostrou que os microserviços ajudam a organizar melhor o sistema e facilitam a escalabilidade. Já o modelo monolítico é mais simples, mas pode ficar mais difícil de manter conforme cresce.

Perguntas

1. Como a refatoração para microserviços afetou a escalabilidade do sistema?

Os microserviços facilitaram a expansão do sistema, pois cada serviço pode crescer separadamente.

2. Quais foram as principais diferenças de performance entre as duas arquiteturas?

O monolítico foi mais simples e rápido em sistemas pequenos, enquanto os microsserviços funcionam melhor em sistemas maiores.

3. Que outros aspectos poderiam ser otimizados para melhorar ainda mais a performance e escalabilidade?

Poderiam ser melhorados o banco de dados, o armazenamento em cache e a comunicação entre serviços.